

Celebal Technologies Internship – Week 8 Assignment

E-Commerce Order Analytics System

Name: Abhishek Raj

Internship: Celebal Technologies Data Engineering Internship

Week: 8

Objective

The objective of this assignment is to develop an end-to-end E-Commerce Order Analytics System using Python, SQL, and SQLite.

The assignment demonstrates the complete data engineering workflow including synthetic data generation, data quality verification, data cleaning, database creation, SQL-based business analytics, command-line reporting, and edge-case validation.

Dataset Description

Four main datasets were used in this assignment.

customers.csv

The customer dataset contains customer information including:

- Customer ID
- Customer Name
- Email

products.csv

The product dataset contains product information including:

- Product ID
 - Product Name
 - Category
 - Unit Price
-

Phase 1 – Data Verification

The raw datasets were verified before performing the cleaning process.

The verification process checked row counts, missing customer IDs, invalid emails, negative quantities, product name inconsistencies, date formats, and referential integrity.

Initial Dataset Sizes

Customers: 500

Products: 500

Orders: 1000

Order Items: 2000

Screenshot

```
=====
PHASE 1 DATA VERIFICATION
=====

1. ROW COUNTS
Customers      : 500
Products       : 500
Orders         : 1000
Order Items    : 2000

2. NULL CUSTOMER IDs IN ORDERS
Missing customer IDs: 100

3. INVALID EMAILS
Invalid emails: 10

4. NEGATIVE QUANTITIES
Negative quantity records: 60

5. PRODUCT NAME INCONSISTENCIES
Products with formatting issues: 40

6. DATE FORMAT CHECK
Wrong-format dates: 40

7. REFERENTIAL INTEGRITY
Order items with invalid order IDs: 0

=====
VERIFICATION COMPLETED
=====
```

Phase 2 – Data Cleaning

The raw datasets were cleaned using Python.

The cleaning process included duplicate detection, email validation, customer ID validation, product name normalization, date conversion, invalid order-reference detection, invalid product-reference detection, and duplicate order-item removal.

Final Dataset Sizes

Customers: 500

Products: 500

Orders: 950

Order Items: 1905

Screenshot

```
=====
PHASE 2 - DATA CLEANING
=====

Loading datasets...
Customers loaded   : 500
Products loaded   : 500
Orders loaded     : 1000
Order items loaded : 2000

-----
1. CLEANING CUSTOMERS
-----
Duplicate customers removed: 0
Invalid emails found: 10
Invalid customer IDs: ['CUST00050', 'CUST00100', 'CUST00150', 'CUST00200', 'CUST00250', 'CUST00300', 'CUST00350', 'CUST00400', 'CUST00450', 'CUST00500']

-----
2. CLEANING PRODUCTS
-----
Duplicate products removed: 0
Product names normalized successfully.
```

3. CLEANING ORDERS

Orders with missing customer ID: 50
Invalid dates after conversion: 0
Duplicate orders removed: 0

4. CLEANING ORDER ITEMS

Invalid order references: 95
Invalid product references: 0
Invalid discount records: 0
Duplicate order items removed: 95

5. SAVING CLEANED DATA

customers_clean.csv saved
products_clean.csv saved
orders_clean.csv saved
order_items_clean.csv saved

=====

PHASE 2 COMPLETED SUCCESSFULLY

=====

Final dataset sizes:

Customers : 500
Products : 500
Orders : 950
Order Items : 1905

Cleaned files saved in:

C:\Users\HP\OneDrive\Desktop\Week-08\data\cleaned

Phase 3 – SQLite Database

The cleaned CSV files were loaded into a SQLite database named ecommerce.db.

The database contains four tables:

- customers
- products
- orders
- order_items

Foreign-key validation confirmed that no foreign-key violations were present.

Database Row Counts

Customers: 500

Products: 500

Orders: 950

Order Items: 1905

Screenshot

```
=====
PHASE 3 - SQLITE DATABASE
=====
```

ROW COUNTS

```
customers      : 500
products       : 500
orders         : 950
order_items    : 1905
```

FOREIGN KEY VALIDATION

No foreign key violations found.

DATABASE TABLES

```
- customers
- order_items
- orders
- products
```

Database created successfully!

Database: C:\Users\HP\OneDrive\Desktop\Week-08\ecommerce.db

Q1. Total Revenue per Category

This query calculates the total revenue generated by each product category.

Screenshot

```
=====
QUERY 1 - TOTAL REVENUE PER CATEGORY
=====
('Clothing', 40077324.67)
('Home', 38443989.53)
('Books', 38306276.16)
('Electronics', 30414655.84)
```

Q2. Top 10 Customers by Total Order Value

This query identifies the top 10 customers based on their total order value.

Screenshot

```
=====
QUERY 2 - TOP 10 CUSTOMERS
=====
('CUST00284', 'Arjun Sharma', 1812715.37)
('CUST00392', 'Karan Kumar', 1706354.51)
('CUST00286', 'Rahul Verma', 1673955.25)
('CUST00267', 'Rahul Agarwal', 1591202.19)
('CUST00183', 'Sneha Singh', 1528265.14)
('CUST00345', 'Amit Agarwal', 1502743.78)
('CUST00215', 'Neha Verma', 1381693.33)
('CUST00204', 'Neha Verma', 1369785.15)
('CUST00195', 'Kavya Patel', 1241325.71)
('CUST00371', 'Aditya Verma', 1238872.49)
```

Q3. Month-wise Order Count

This query calculates the number of orders placed during each month.

Screenshot

```
=====
QUERY 3 - MONTH-WISE ORDER COUNT
=====
('2025-09', 30)
('2025-10', 35)
('2025-11', 29)
('2025-12', 39)
('2026-01', 34)
('2026-02', 26)
('2026-03', 38)
('2026-04', 33)
('2026-05', 35)
('2026-06', 32)
('2026-07', 25)
('2026-08', 1)
```

Q4. Customers Who Placed Orders but Never Had a Delivered Item

This query identifies customers who placed orders but never had a delivered item.

Total Customers: 294

Screenshot

```
=====
Q4 - CUSTOMERS WHO PLACED ORDERS BUT NEVER HAD A DELIVERED ITEM
=====
Total customers: 294

Sample results:
('CUST00001', 'Vikas Verma')
('CUST00003', 'Simran Verma')
('CUST00004', 'Nikhil Malhotra')
('CUST00005', 'Sneha Sharma')
('CUST00006', 'Neha Kumar')
('CUST00007', 'Rahul Malhotra')
('CUST00009', 'Karan Agarwal')
('CUST00010', 'Aditya Singh')
('CUST00011', 'Ankit Gupta')
('CUST00012', 'Aditya Mehta')
```

Q5. Products with More Returns Than Purchases

This query identifies products where the number of returned items is greater than the number of purchased items.

Screenshot

```
=====
Q5 - PRODUCTS WITH MORE RETURNS THAN PURCHASES
=====
('PROD00126', 'Sofa', 3, 11)
('PROD00194', 'Table', 0, 2)
('PROD00403', 'History Book', 0, 1)
```

Q6. Return Rate per Category

This query calculates the return rate for each product category.

Screenshot

```
=====
Q6 - RETURN RATE PER CATEGORY
=====
```

```
('Home', 43, 2465, 1.74)
('Clothing', 29, 2783, 1.04)
('Books', 25, 2859, 0.87)
('Electronics', 13, 2163, 0.6)
```

Q7. Running Total of Revenue per Region

This query calculates daily revenue and the running cumulative revenue for each region using window functions.

Total Rows: 757

Screenshot

```
=====
QUERY 7 - RUNNING TOTAL BY REGION
=====
```

```
('CENTRAL', '2024-01-12', 182491.34, 182491.34)
('CENTRAL', '2024-01-27', 68619.68, 251111.02)
('CENTRAL', '2024-02-03', 237533.29, 488644.31)
('CENTRAL', '2024-02-04', 227483.47, 716127.78)
('CENTRAL', '2024-02-12', 186651.86, 902779.64)
('CENTRAL', '2024-02-20', 352065.97, 1254845.61)
('CENTRAL', '2024-02-27', 65147.58, 1319993.19)
('CENTRAL', '2024-03-03', 580338.09, 1900331.28)
('CENTRAL', '2024-03-06', 172967.12, 2073298.4)
('CENTRAL', '2024-03-15', 41536.92, 2114835.32)
('CENTRAL', '2024-03-21', 286261.44, 2401096.76)
('CENTRAL', '2024-03-25', -38227.23, 2362869.53)
('CENTRAL', '2024-03-28', 414287.41, 2777156.94)
('CENTRAL', '2024-04-04', 83728.65, 2860885.59)
('CENTRAL', '2024-04-15', 270106.9, 3130992.49)
('CENTRAL', '2024-04-16', 377820.61, 3508813.1)
('CENTRAL', '2024-04-21', 158983.94, 3667797.04)
('CENTRAL', '2024-05-12', 80052.32, 3747849.36)
('CENTRAL', '2024-05-13', 45954.12, 3793803.48)
('CENTRAL', '2024-05-15', 171003.8, 3964807.28)
```

Total rows: 757

```
=====
QUERY 7 COMPLETED SUCCESSFULLY
=====
```

Q8. Product Revenue Rank by Category

This query ranks products according to their revenue within each category.

Total Rows: 486

Screenshot

```
=====
QUERY 8 - PRODUCT REVENUE RANK BY CATEGORY
=====
```

```
('Books', 'Science Book', 1152649.86, 1)
('Books', 'Science Book', 1140955.26, 2)
('Books', 'Science Book', 998223.2, 3)
('Books', 'History Book', 820654.67, 4)
('Books', 'Math Book', 730810.95, 5)
('Books', 'Biography', 713435.36, 6)
('Books', 'Math Book', 694304.64, 7)
('Books', 'Programming Book', 656294.39, 8)
('Books', 'History Book', 650209.71, 9)
('Books', 'Math Book', 633327.52, 10)
('Books', 'History Book', 607302.26, 11)
('Books', 'Novel', 598436.66, 12)
('Books', 'Novel', 595295.78, 13)
('Books', 'Programming Book', 586919.27, 14)
('Books', 'History Book', 583927.67, 15)
('Books', 'Novel', 559038.33, 16)
('Books', 'Biography', 557794.79, 17)
('Books', 'Biography', 556855.34, 18)
('Books', 'Programming Book', 556659.06, 19)
('Books', 'Math Book', 547901.24, 20)
```

```
Total rows: 486
```

```
=====
QUERY 8 COMPLETED SUCCESSFULLY
=====
```

Q9. Customer Order Gap Analysis

This query uses the LAG() window function to calculate the gap between consecutive customer orders.

Total Rows: 812

Screenshot

```
=====
QUERY 9 - LAG ANALYSIS / CUSTOMER ORDER GAPS
=====
```

```
('CUST00001', '2024-05-01', None, None, 'At Risk')
('CUST00001', '2024-12-05', '2024-05-01', 218, 'At Risk')
('CUST00001', '2026-01-13', '2024-12-05', 404, 'At Risk')
('CUST00001', '2026-02-16', '2026-01-13', 34, 'At Risk')
('CUST00003', '2024-02-08', None, None, 'At Risk')
('CUST00003', '2024-11-14', '2024-02-08', 280, 'At Risk')
('CUST00003', '2024-11-16', '2024-11-14', 2, 'At Risk')
('CUST00004', '2025-01-19', None, None, 'At Risk')
('CUST00004', '2025-02-23', '2025-01-19', 35, 'At Risk')
('CUST00004', '2025-10-03', '2025-02-23', 222, 'At Risk')
('CUST00005', '2026-02-23', None, None, 'At Risk')
('CUST00005', '2026-05-22', '2026-02-23', 88, 'At Risk')
('CUST00008', '2024-06-20', None, None, 'At Risk')
('CUST00008', '2024-12-17', '2024-06-20', 180, 'At Risk')
('CUST00010', '2024-12-21', None, None, 'At Risk')
('CUST00010', '2025-11-23', '2024-12-21', 337, 'At Risk')
('CUST00011', '2025-12-06', None, None, 'At Risk')
('CUST00011', '2026-06-04', '2025-12-06', 180, 'At Risk')
('CUST00013', '2024-07-02', None, None, 'At Risk')
('CUST00013', '2026-03-29', '2024-07-02', 635, 'At Risk')
```

```
Total rows: 812
```

```
=====
QUERY 9 COMPLETED SUCCESSFULLY
=====
```

Q10. Monthly Customer Revenue Segmentation

This query segments customers based on their monthly revenue contribution.

Total Rows: 75

Screenshot

```
=====
QUERY 10 - MONTHLY CUSTOMER REVENUE SEGMENTATION
=====
('2024-01', 'High', 15)
('2024-01', 'Medium', 2)
('2024-01', 'Low', 1)
('2024-02', 'High', 30)
('2024-02', 'Low', 1)
('2024-03', 'High', 18)
('2024-03', 'Low', 2)
('2024-04', 'High', 25)
('2024-04', 'Medium', 1)
('2024-04', 'Low', 3)
('2024-05', 'High', 20)
('2024-06', 'High', 27)
('2024-07', 'High', 24)
('2024-08', 'High', 21)
('2024-08', 'Medium', 3)
('2024-08', 'Low', 1)
('2024-09', 'High', 16)
('2024-09', 'Low', 2)
('2024-10', 'High', 15)
('2024-10', 'Medium', 2)

Total rows: 75
=====
QUERY 10 COMPLETED SUCCESSFULLY
=====
```

Q11. Customer Lifetime Value Quartiles

This query calculates customer lifetime value and assigns customers to quartile-based segments.

Total Rows: 405

Screenshot

```
=====
QUERY 11 - CUSTOMER LIFETIME VALUE QUARTILES
=====
('CUST00284', 1812715.37, 1, 'Platinum')
('CUST00392', 1706354.51, 1, 'Platinum')
('CUST00286', 1673955.25, 1, 'Platinum')
('CUST00267', 1591202.19, 1, 'Platinum')
('CUST00183', 1528265.14, 1, 'Platinum')
('CUST00345', 1502743.78, 1, 'Platinum')
('CUST00215', 1381693.33, 1, 'Platinum')
('CUST00204', 1369785.15, 1, 'Platinum')
('CUST00195', 1241325.71, 1, 'Platinum')
('CUST00371', 1238872.49, 1, 'Platinum')
('CUST00184', 1212656.1, 1, 'Platinum')
('CUST00219', 1206937.24, 1, 'Platinum')
('CUST00188', 1126866.37, 1, 'Platinum')
('CUST00336', 1114224.67, 1, 'Platinum')
('CUST00488', 1087464.39, 1, 'Platinum')
('CUST00414', 1085596.67, 1, 'Platinum')
('CUST00393', 1085272.59, 1, 'Platinum')
('CUST00486', 1084003.44, 1, 'Platinum')
('CUST00199', 1040383.51, 1, 'Platinum')
('CUST00325', 1037888.68, 1, 'Platinum')

Total rows: 405

=====
QUERY 11 COMPLETED SUCCESSFULLY
=====
```

Q12. Year-over-Year Revenue Comparison

This query compares monthly revenue across different years and calculates the percentage change.

Total Rows: 32

Screenshot

```
=====
QUERY 12 - YEAR-OVER-YEAR REVENUE COMPARISON
=====
('2024', '01', 2574887.59, None, None)
('2024', '02', 5916344.81, None, None)
('2024', '03', 3633417.68, None, None)
('2024', '04', 4080166.38, None, None)
('2024', '05', 4411009.61, None, None)
('2024', '06', 6057805.01, None, None)
('2024', '07', 5380360.44, None, None)
('2024', '08', 4294536.34, None, None)
('2024', '09', 3460953.88, None, None)
('2024', '10', 3234768.87, None, None)
('2024', '11', 5582208.72, None, None)
('2024', '12', 6283524.45, None, None)
('2025', '01', 5081197.12, 2574887.59, 97.34)
('2025', '02', 2493473.2, 5916344.81, -57.85)
('2025', '03', 2400610.68, 3633417.68, -33.93)
('2025', '04', 5625575.64, 4080166.38, 37.88)
('2025', '05', 3545149.46, 4411009.61, -19.63)
('2025', '06', 5309070.76, 6057805.01, -12.36)
('2025', '07', 4257348.87, 5380360.44, -20.87)
('2025', '08', 4464419.72, 4294536.34, 3.96)

Total rows: 32
QUERY 12 COMPLETED SUCCESSFULLY
```

Q13. First and Last Purchased Category

This query identifies the first and last product category purchased by each customer.

Total Rows: 405

Screenshot

```
=====
QUERY 13 - FIRST AND LAST PURCHASED CATEGORY
=====
('CUST00001', 'Electronics', 'Home', 'Yes')
('CUST00003', 'Books', 'Home', 'Yes')
('CUST00004', 'Books', 'Clothing', 'Yes')
('CUST00005', 'Home', 'Electronics', 'Yes')
('CUST00006', 'Clothing', 'Clothing', 'No')
('CUST00007', 'Electronics', 'Electronics', 'No')
('CUST00008', 'Clothing', 'Clothing', 'No')
('CUST00009', 'Home', 'Home', 'No')
('CUST00010', 'Clothing', 'Clothing', 'No')
('CUST00011', 'Home', 'Electronics', 'Yes')
('CUST00012', 'Electronics', 'Electronics', 'No')
('CUST00013', 'Home', 'Home', 'No')
('CUST00014', 'Electronics', 'Electronics', 'No')
('CUST00015', 'Home', 'Home', 'No')
('CUST00016', 'Home', 'Home', 'No')
('CUST00017', 'Home', 'Home', 'No')
('CUST00018', 'Electronics', 'Electronics', 'No')
('CUST00021', 'Clothing', 'Books', 'Yes')
('CUST00023', 'Clothing', 'Clothing', 'No')
('CUST00024', 'Electronics', 'Books', 'Yes')

Total rows: 405
QUERY 13 COMPLETED SUCCESSFULLY
```

Q14. Cumulative Revenue Distribution

This query calculates cumulative customer revenue and the percentage contribution to overall revenue.

Total Rows: 405

Screenshot

```
=====
QUERY 14 - CUMULATIVE REVENUE DISTRIBUTION
=====
('CUST00284', 1812715.37, 1812715.37, 1.23)
('CUST00392', 1706354.51, 3519069.88, 2.39)
('CUST00286', 1673955.25, 5193025.14, 3.53)
('CUST00267', 1591202.19, 6784227.33, 4.61)
('CUST00183', 1528265.14, 8312492.47, 5.65)
('CUST00345', 1502743.78, 9815236.25, 6.67)
('CUST00215', 1381693.33, 11196929.58, 7.6)
('CUST00204', 1369785.15, 12566714.72, 8.53)
('CUST00195', 1241325.71, 13808040.43, 9.38)
('CUST00371', 1238872.49, 15046912.92, 10.22)
('CUST00184', 1212656.1, 16259569.01, 11.04)
('CUST00219', 1206937.24, 17466506.25, 11.86)
('CUST00188', 1126866.37, 18593372.62, 12.63)
('CUST00336', 1114224.67, 19707597.28, 13.38)
('CUST00488', 1087464.39, 20795061.67, 14.12)
('CUST00414', 1085596.67, 21880658.34, 14.86)
('CUST00393', 1085272.59, 22965930.93, 15.6)
('CUST00486', 1084003.44, 24049934.37, 16.33)
('CUST00199', 1040383.51, 25090317.87, 17.04)
('CUST00325', 1037888.68, 26128206.55, 17.75)

Total rows: 405
QUERY 14 COMPLETED SUCCESSFULLY
```

Q15. Cohort Retention Analysis

This query performs cohort retention analysis by tracking customer activity across different months.

Total Rows: 32

Screenshot

```
=====
QUERY 15 - COHORT RETENTION ANALYSIS
=====
('2023-01', 14, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-02', 10, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-03', 19, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-04', 24, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-05', 8, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-06', 17, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-07', 12, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-08', 22, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-09', 15, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-10', 18, 0, 0, 0, 1, 0.0, 0.0, 0.0, 5.56)
('2023-11', 10, 0, 0, 0, 0, 0.0, 0.0, 0.0, 0.0)
('2023-12', 13, 0, 0, 2, 1, 0.0, 0.0, 15.38, 7.69)
('2024-01', 14, 0, 1, 2, 1, 0.0, 7.14, 14.29, 7.14)
('2024-02', 12, 0, 1, 0, 0, 0.0, 8.33, 0.0, 0.0)
('2024-03', 16, 0, 3, 0, 1, 0.0, 18.75, 0.0, 6.25)
('2024-04', 23, 0, 2, 1, 2, 0.0, 8.7, 4.35, 8.7)
('2024-05', 16, 1, 0, 3, 1, 6.25, 0.0, 18.75, 6.25)
('2024-06', 12, 1, 1, 2, 1, 8.33, 8.33, 16.67, 8.33)
('2024-07', 13, 1, 1, 1, 0, 7.69, 7.69, 7.69, 0.0)
('2024-08', 21, 0, 1, 0, 3, 0.0, 4.76, 0.0, 14.29)

Total rows: 32
=====
QUERY 15 COMPLETED SUCCESSFULLY
=====
```


Q16. Self Join with Window Function

This query analyzes consecutive customer orders using the LAG() window function.

Total Rows: 950

Screenshot

```
=====
QUERY 16 - SELF JOIN WITH WINDOW FUNCTION
=====
('ORD000923', 'CUST00001', '2024-05-01', None, None, None)
('ORD000117', 'CUST00001', '2024-12-05', 'ORD000923', '2024-05-01', 218)
('ORD000764', 'CUST00001', '2026-01-13', 'ORD000117', '2024-12-05', 404)
('ORD000053', 'CUST00001', '2026-02-16', 'ORD000764', '2026-01-13', 34)
('ORD000271', 'CUST00003', '2024-02-08', None, None, None)
('ORD000959', 'CUST00003', '2024-11-14', 'ORD000271', '2024-02-08', 280)
('ORD000471', 'CUST00003', '2024-11-16', 'ORD000959', '2024-11-14', 2)
('ORD000543', 'CUST00004', '2025-01-19', None, None, None)
('ORD000051', 'CUST00004', '2025-02-23', 'ORD000543', '2025-01-19', 35)
('ORD000428', 'CUST00004', '2025-10-03', 'ORD000051', '2025-02-23', 222)
('ORD000670', 'CUST00005', '2026-02-23', None, None, None)
('ORD000313', 'CUST00005', '2026-05-22', 'ORD000670', '2026-02-23', 88)
('ORD000489', 'CUST00006', '2025-11-09', None, None, None)
('ORD000022', 'CUST00007', '2025-12-25', None, None, None)
('ORD000706', 'CUST00008', '2024-06-20', None, None, None)
('ORD000809', 'CUST00008', '2024-12-17', 'ORD000706', '2024-06-20', 180)
('ORD000528', 'CUST00009', '2024-07-26', None, None, None)
('ORD000828', 'CUST00010', '2024-12-21', None, None, None)
('ORD000274', 'CUST00010', '2025-11-23', 'ORD000828', '2024-12-21', 337)
('ORD000717', 'CUST00011', '2025-12-06', None, None, None)

Total rows: 950
QUERY 16 COMPLETED SUCCESSFULLY
```

Phase 5 – Command-Line Reporting

A Python-based command-line reporting system was developed to generate Daily, Weekly, and Monthly reports.

Each report provides:

- Total Orders
- Total Revenue
- Unique Customers
- Previous Period Comparison
- Order Change
- Revenue Change
- Top 3 Products

Screenshot

```
E-COMMERCE ORDER ANALYTICS CLI
=====

Available report types:
1. Daily
2. Weekly
3. Monthly

Enter report type (daily/weekly/monthly): monthly
Enter start date (YYYY-MM-DD): 2026-01-01
Enter end date (YYYY-MM-DD): 2026-01-31

=====
E-COMMERCE ORDER ANALYTICS REPORT
=====

Report Type      : MONTHLY
Date Range       : 2026-01-01 to 2026-01-31
Previous Period  : 2025-12-01 to 2025-12-31

-----
SUMMARY
-----
Total Orders     : 29
Total Revenue    : ₹ 5289463.27
Unique Customers : 27

-----
PREVIOUS PERIOD COMPARISON
-----
Previous Orders  : 34
Previous Revenue : ₹ 5934297.92
Order Change     : -14.71%
Revenue Change   : -10.87%

-----
TOP 3 PRODUCTS
-----
1. Dress - ₹480641.27
2. Jeans - ₹363396.24
3. Headphones - ₹301150.83
```

E-COMMERCE ORDER ANALYTICS CLI

Available report types:

1. Daily
2. Weekly
3. Monthly

Enter report type (daily/weekly/monthly): daily

Enter start date (YYYY-MM-DD): 2026-01-15

Enter end date (YYYY-MM-DD): 2026-01-15

E-COMMERCE ORDER ANALYTICS REPORT

Report Type : DAILY
Date Range : 2026-01-15 to 2026-01-15
Previous Period : 2026-01-14 to 2026-01-14

SUMMARY

Total Orders : 0
Total Revenue : ₹ 0.0
Unique Customers : 0

PREVIOUS PERIOD COMPARISON

Previous Orders : 1
Previous Revenue : ₹ 136540.38
Order Change : -100.00%
Revenue Change : -100.00%

TOP 3 PRODUCTS

No products found for this period.

REPORT GENERATED SUCCESSFULLY

```
=====
E-COMMERCE ORDER ANALYTICS CLI
=====
```

Available report types:

1. Daily
2. Weekly
3. Monthly

Enter report type (daily/weekly/monthly): weekly

Enter start date (YYYY-MM-DD): 2026-01-01

Enter end date (YYYY-MM-DD): 2026-01-07

```
=====
E-COMMERCE ORDER ANALYTICS REPORT
=====
```

Report Type : WEEKLY
Date Range : 2026-01-01 to 2026-01-07
Previous Period : 2025-12-25 to 2025-12-31

```
-----
SUMMARY
-----
```

Total Orders : 4
Total Revenue : ₹ 202981.81
Unique Customers : 4

```
-----
PREVIOUS PERIOD COMPARISON
-----
```

Previous Orders : 6
Previous Revenue : ₹ 1281233.67
Order Change : -33.33%
Revenue Change : -84.16%

```
-----
TOP 3 PRODUCTS
-----
```

1. Headphones - ₹85529.79
2. Sweater - ₹52644.04
3. Pillow - ₹20773.73

Phase 6 – Edge Case Testing

The system was tested against invalid and unusual scenarios.

Test 1 – Invalid Order ID

Status: PASSED

Invalid order ID was rejected by the foreign-key constraint.

Test 2 – Discount Greater Than 100%

Status: PASSED

Invalid discount detected: 120%.

Test 3 – Zero Quantity

Status: PASSED

Zero quantity was detected and treated as an edge case.

Test 4 – Future Order Date

Status: PASSED

Future order date detected: 2099-12-31.

Screenshot

```
=====
PHASE 6 - EDGE CASE TESTING
=====

-----
TEST 1 - INVALID ORDER ID
-----
PASSED
Invalid order_id rejected by foreign key constraint.

-----
TEST 2 - DISCOUNT GREATER THAN 100%
-----
PASSED
Invalid discount detected: 120 %

-----
TEST 3 - ZERO QUANTITY
-----
PASSED
Zero quantity detected and treated as an edge case.

-----
TEST 4 - FUTURE ORDER DATE
-----
PASSED
Future order date detected: 2099-12-31

=====
PHASE 6 EDGE CASE TESTING COMPLETED
=====
```

Conclusion

This assignment successfully demonstrated an end-to-end E-Commerce Order Analytics System using Python, SQL, and SQLite.

The project covered data generation, data verification, data cleaning, database creation, advanced SQL analysis, command-line reporting, and edge-case testing.

The implementation provided practical experience with SQL aggregations, joins, subqueries, CTEs, window functions, ranking, customer segmentation, cohort analysis, cumulative analysis, and self-joins.

All major phases of the project were successfully completed, including the 16 SQL analytical queries, CLI reporting system, and four edge-case validation tests.

References

- [Python Documentation](#)
- [SQLite Documentation](#)
- [Pandas Documentation](#)
- [SQL Documentation](#)
- [Celebal Technologies Internship Material](#)